Colecțiile din .NET — Exerciții

Ștefan · pornești mereu de la interfață: `IList<string> nume = new List<string>();`

Unde scrii: `Exercitii/Sectiune1ICollection.cs` … `Sectiune5QueueStack.cs`, `Sectiune6Alegere.cs`: metoda cu numele din tabel (datele de pornire sunt deja declarate); `Main` le cheamă pe toate. Cărțile metodelor: **COLECTII.pdf**. Exemple de rulat și citit: `Exemple/`.
Ieșirea: colecția o afișezi cu `string.Join(", ", c)`, un `bool` sau un număr cum sunt; excepția o prinzi cu `try/catch (Exception e)` și afișezi `e.GetType().Name`. Celula goală = o notezi tu după ce rulezi.

1. ICollection<T> → List, HashSet, LinkedList

```
ICollection<string> c = new List<string>();
```

#	Cerința	Ieșire
1.1	**`AdaugaCuDuplicate`** `c` e goală. `Add` pe rând Ana, Ion, Maria, Ana; afișează `Count`.	`4`
1.2	**`VerificaContine`** `c` = [Ana, Ion, Maria]. Afișează `Contains("Ion")`, apoi `Contains("Dan")`.	`True` `False`
1.3	**`ScoateIonSiZoe`** `c` = [Ana, Ion, Maria]. `Remove("Ion")`, apoi `Remove("Zoe")`: afișează ce întoarce fiecare, apoi colecția.	
1.4	**`GolesteColectia`** `c` = [Ana, Ion, Maria]. `Clear()`, apoi afișează `Count`.	`0`
1.5	**`DeCeInterfataICollection`** Scrie `Afiseaza(ICollection<int> c)`: afișează `Count`, „: ” și elementele. Cheam-o cu `List`, `HashSet`, `LinkedList`, toate din `v`. Compară cele 3 ieșiri.	

2. IList<T> → List, T[]

```
IList<string> l = new List<string>();
```

#	Cerința	Ieșire
2.1	**`PuneDanPePrimaPozitie`** `l` = [Ana, Ion, Maria]. Pune „Dan” pe prima poziție (`Insert`); afișează lista.	`Dan, Ana, Ion, Maria`
2.2	**`PozitiaLuiMariaSiZoe`** `l` = [Ana, Ion, Maria]. Afișează `IndexOf("Maria")`, apoi `IndexOf("Zoe")`.	`2` `-1`
2.3	**`InlocuiesteIonCuEvaSiScoatePrimul`** `l` = [Ana, Ion, Maria]. `l[1] = "Eva"`, apoi `RemoveAt(0)`; afișează lista.	`Eva, Maria`
2.4	**`CitesteDupaUltimaPozitie`** `l` = [Ana, Ion, Maria]. Citește `l[3]` într-un `try/catch`; afișează numele excepției.	
2.5	**`DeCeInterfataIList`** Scrie `Mareste(IList<int> l)` (`Add(4)`, apoi afișează lista) și `Schimba(IList<int> l)` (`l[0] = 9`, apoi afișează lista). Cheam-o pe fiecare cu un `List<int>` și cu un `int[]`, ambele {1, 2, 3}; `Mareste` pe tablou, în `try/catch`.	

3. ISet<T> → HashSet, SortedSet

```
ISet<string> s = new HashSet<string>();
```

#	Cerința	Ieșire
3.1	**`AdaugaAnaIonAnaMaria`** `s` e goală. `Add` pe rând Ana, Ion, Ana, Maria: afișează ce întoarce fiecare `Add`.	

3.2	**ReuniuneCuIonSiEva** `s` = {Ana, Ion, Maria}. `UnionWith` cu [Ion, Eva]; afișează mulțimea.	`Ana, Ion, Maria, Eva`
3.3	**IntersectieCuIonMariaZoe** `s` = {Ana, Ion, Maria}. `IntersectWith` cu [Ion, Maria, Zoe]; afișează mulțimea.	`Ion, Maria`
3.4	**ScoateIonSiNumara** `s` = {Ana, Ion, Maria}. `Remove("Ion")`; afișează `Contains("Ion")`, apoi `Count`.	`False` `2`
3.5	**DeCeInterfataISet** Scrie `AfiseazaMultime(ISet<string> s)`: afișează `Count`, „: ” și elementele. Cheam-o cu un `HashSet` și cu un `SortedSet`, ambele din `v`. Compară.	

4. IDictionary<K, V> → Dictionary, SortedDictionary, SortedList

```
IDictionary<string, int> d = new Dictionary<string, int>();
```

#	Cerința	Ieșire
4.1	**AdaugaSiSuprascrie** `d` e goală. `d["Ana"] = 9`, `Add("Ion", 7)`, apoi `d["Ion"] = 10`; afișează `Count` și `d["Ion"]`.	`2` `10`

4.2	**CautaCuTryGetValue** `d` = {Ana: 9, Ion: 7}. `TryGetValue("Dan", out v)`: afișează bool-ul și `v`; la fel pentru „Ana”.	`False`
		`0`
		`True`
		`9`

4.3	**AdaugaAnaDinNouSiCiteste Dan** `d` = {Ana: 9, Ion: 7}. `Add("Ana", 5)`, apoi `d["Dan"]`, fiecare în `try/catch`; afișează numele excepției.	

4.4	**ScoateAnaDeDouaOri** `d` = {Ana: 9, Ion: 7}. `Remove("Ana")` de două ori, apoi `ContainsKey("Ion")`; afișează cele 3 rezultate.	`True`
		`False`
		`True`

4.5	**DeCeInterfataIDictionary** Scrie `AfiseazaDictionar(IDictionary<string, int> d)`: `string.Join(", ", d)`. Cheam-o cu `Dictionary`, `SortedDictionary`, `SortedList`, toate din `date`. Compară.	

5. Queue<T> și Stack<T>

Nu au o interfață comună cu `List` (nu au `Add` și nici poziții), deci declari direct clasa.

```
Queue<string> q = new Queue<string>();
Stack<string> s = new Stack<string>();
```

#	Cerința	Ieșire

5.1	**ServesteUnClient** `q` = [Ana, Ion, Maria] (primul = Ana). `Dequeue()`, apoi `Peek()`, apoi `Count`; afișează cele 3 rezultate.	`Ana`
		`Ion`
		`2`

5.2	**StivaPopPeekSiParcurgere** `s`: `Push` pe rând Ana, Ion, Maria. `Pop()`, apoi `Peek()`, apoi toată stiva cu `string.Join`; afișează cele 3.	`Maria`
		`Ion`
		`Ion, Ana`

5.3	**ScoateToataCoada** `q` = [Ana, Ion, Maria]. Cu `while (q.Count > 0)` scoate și afișează fiecare client.	`Ana`
		`Ion`
		`Maria`

5.4	**ScoateDinGol** O coadă goală: `Dequeue()`; o stivă goală: `Pop()`. Fiecare în `try/catch`; afișează numele excepției.	

6. Ce colecție alegi — aplici

Pentru fiecare situație scrii colecția pe care o alegi (coloana a doua), codul în metoda ei, apoi notezi ieșirea (un rând, valorile separate prin „, ”).

#	Situația	Colecția aleasă	Ieșire
6.1	`SituatiaVizitatori` La intrare trec, pe rând: Ana, Ion, Ana, Maria, Ion. Afișează câți vizitatori **diferiți** au fost.		
6.2	`SituatiaIstoric` Butonul „Înapoi”: ai deschis paginile A, B, C (în ordinea asta). Apeși „Înapoi” de două ori: afișează paginile în ordinea întoarcerii, pe un rând.		
6.3	`SituatiaTelefoane` Agenda: Ana → 0711, Ion → 0722. Caută „Ion”, apoi „Dan”; afișează numerele pe un rând (cuvântul `lipsa` dacă nu există).		
6.4	`SituatiaTemperaturi` Exact 5 zile, temperaturile 12, 8, 15, 9, 11. Sortează-le; afișează lista sortată și, după ea, poziția lui 15 în ea.		
6.5	`SituatiaGhiseu` Clienții la ghișeu, serviți în ordinea sosirii: Ana, Ion, Maria. Servește 2; afișează-i în ordinea servirii, pe un rând.		
6.6	`SituatiaNumere` Numerele 5, 3, 8, 3, 1. Afișează cel mai mic, cel mai mare, apoi toate numerele **fără dubluri**, crescător; totul pe un rând.		
6.7	`SituatiaNote` Notele 9, 6, 10, 7, în ordinea asta, pe poziții. Pune 8 pe poziția 1, scoate elementul de pe poziția 3, sortează; afișează lista pe un rând.		
6.8	`SituatiaScoruri` Scorurile Zoe = 5, Ana = 9, Ion = 7, cu numele mereu în ordine alfabetică. Afișează **a doua** cheie în ordine.		
6.9	`SituatiaMetrou` Pasagerii la coadă: Ion, Maria. Pune-o pe Ana la început și pe Dan la sfârșit, apoi scoate ultimul; afișează rândul pe un rând.		
6.10	`SituatiaClasament` Scorurile Zoe = 5, Ana = 9, Ion = 7, căutate după nume. Afișează-le în ordinea alfabetică a numelor, ca `nume=scor`, pe un rând.		